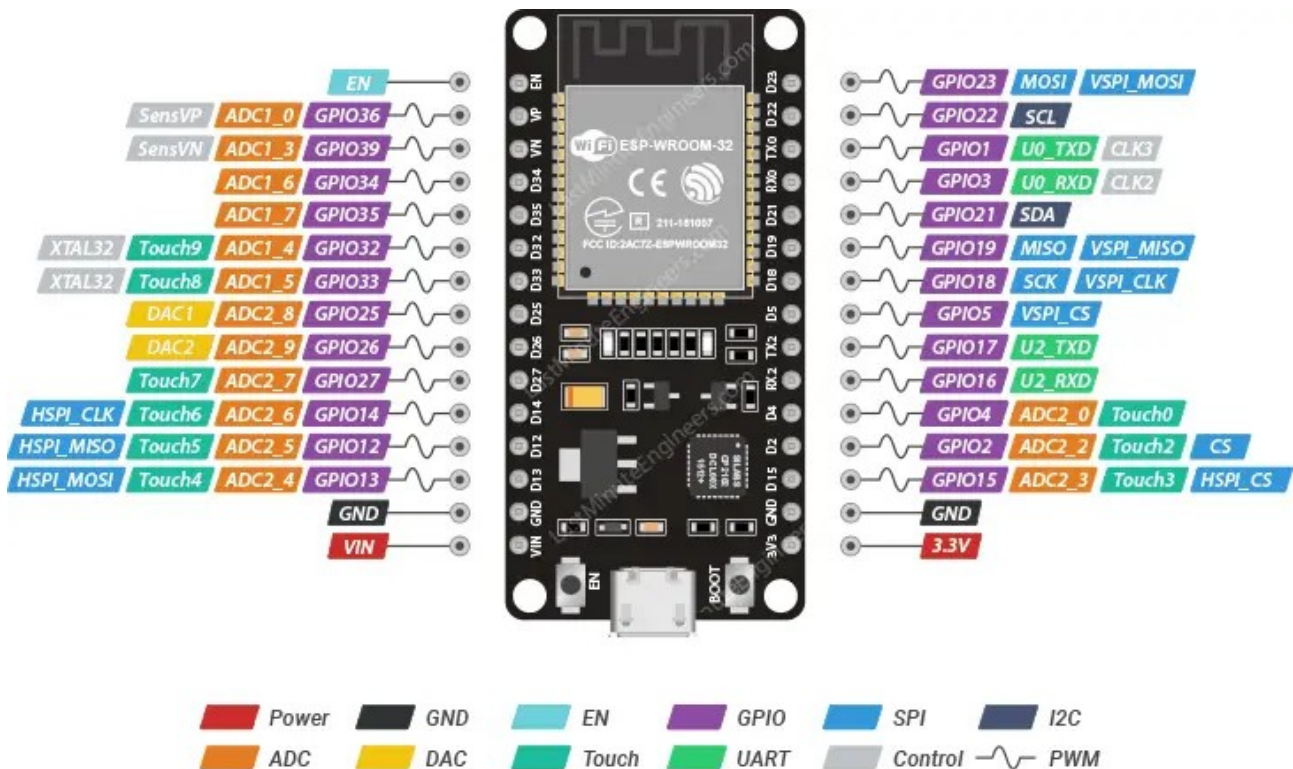


Proposta de Ambiente IoT e suporte aos projetos.

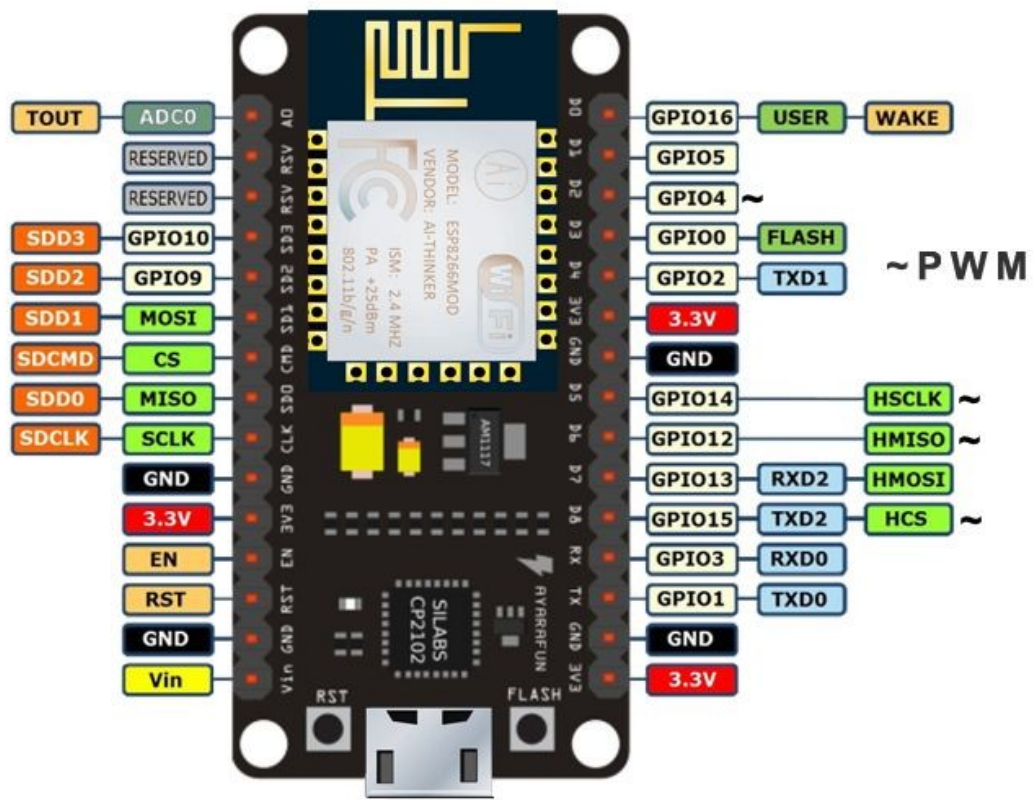
Temos atualmente a existência de 2 modelos para trabalhar o ESP8266 e o ESP32

Pinout – ESP32



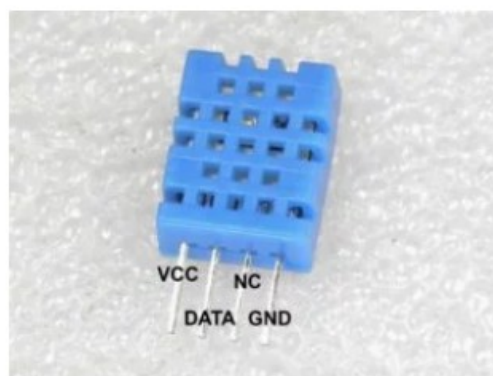
ESP32 Dev. Board Pinout

Pinout – ESP8266



Pinout do sensor DHT11

- VCC = 3,5 a 5V
- DATA = comunicação de dados
- NC = sem conexão
- GND = terra



Sensor DHT11 – Pinout

Baixe e instale a versão mais recente do Arduino IDE:

- <https://www.arduino.cc/en/software>

Preparar para Arduino Uno

1. **Conecte o Arduino Uno via USB.**
2. Vá em **Ferramentas > Placa** e selecione "**Arduino Uno**".
3. Vá em **Ferramentas > Porta** e selecione a porta correta (ex: COM3).
4. **Drivers geralmente são instalados automaticamente** no Windows. Se não:
 - Baixe o driver CH340/ATmega16U2 conforme seu modelo:
 - CH340: <https://sparks.gogo.co.nz/ch340.html>
 - CP2102: <https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers>

Instalar Suporte ao ESP32

1. Vá em **Arquivo > Preferências**.
2. Em "**URLs adicionais para Gerenciador de Placas**", cole:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

3. Vá em **Ferramentas > Placa > Gerenciador de Placas**.
4. Pesquise por **esp32** e instale o pacote da **Espressif Systems**.

Selecionar a placa e porta

- Vá em **Ferramentas > Placa** e selecione o modelo **ESP32 Dev Module** (LilyGO).

Incluir Biblioteca > Gerenciar Bibliotecas.

- Pesquise por **DHT sensor library**.
- Instale a biblioteca da **Adafruit**.
- Também instale: **Adafruit Unified Sensor**

Instalando o Módulo Wi-Fi

Instalar Suporte à Placa

Se for ESP8266 (NodeMCU):

1. Vá em **Arquivo > Preferências**.
2. Em "**URLs adicionais para Gerenciador de Placas**", adicione:

http://arduino.esp8266.com/stable/package_esp8266com_index.json

3. Vá em **Ferramentas > Placa > Gerenciador de Placas**.
4. Procure por **ESP8266** e instale o pacote da **ESP8266 Community**.

Se for ESP32 (LilyGO):

1. Vá em **Arquivo > Preferências**.
2. Adicione esta URL:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

3. Vá em **Ferramentas > Placa > Gerenciador de Placas**.
4. Procure por **ESP32** e instale o pacote da **Espressif Systems**.

Selecionar a Placa e Porta

- Vá em **Ferramentas > Placa** e selecione:
 - **NodeMCU 1.0** (para ESP8266)
 - **ESP32 Dev Module** ou o modelo LilyGO (para ESP32)
- Vá em **Ferramentas > Porta** e selecione a porta COM correta.

Bibliotecas Wi-Fi

As bibliotecas Wi-Fi vêm **integradas** com os pacotes da placa:

Placa	Biblioteca Wi-Fi
ESP8266	<code>#include <ESP8266WiFi.h></code>
ESP32	<code>#include <WiFi.h></code>

Código de Exemplo para Arduino Uno e ESP32

Para ambos, use:

```
#include "DHT.h"

#define DHTPIN 4           // Pino digital onde o DHT11 está conectado
#define DHTTYPE DHT11     // Modelo do sensor

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();

  if (isnan(h) || isnan(t)) {
    Serial.println("Falha na leitura do sensor DHT!");
    return;
  }

  Serial.print("Umidade: ");
  Serial.print(h);
  Serial.print(" %\t");
  Serial.print("Temperatura: ");
  Serial.print(t);
  Serial.println(" *C");

  delay(2000);
}
```

Código de Teste (Conectar ao Wi-Fi)

Para ESP8266:

```
#include <WiFi.h>

const char* ssid = "NOME SSID";
const char* password = "senha"; // substitua aqui

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("Conectando ao Wi-Fi...");

  WiFi.disconnect(true); // Remove redes salvas
  delay(1000);

  Serial.println("Procurando redes...");
  int redes = WiFi.scanNetworks();
  for (int i = 0; i < redes; i++) {
```

```

Serial.print("Rede encontrada: ");
Serial.print(WiFi.SSID(i));
Serial.print(" | Sinal: ");
Serial.println(WiFi.RSSI(i));
}

WiFi.begin(ssid, password);

int tentativas = 0;
while (WiFi.status() != WL_CONNECTED && tentativas < 20) {
  delay(500);
  Serial.print(".");
  tentativas++;
}

if (WiFi.status() == WL_CONNECTED) {
  Serial.println("\nConectado com sucesso!");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());
} else {
  Serial.println("\nFalha ao conectar. Verifique SSID/senha ou roteador.");
}
}

void loop() {}

```

Para testar a conectividade, experimento pingar (comando ping) para o IP que foi atribuído ao ESP32 ou 8266.

Para conectar no Blynk.IO

Vá para <https://blynk.cloud>

Clique em “Templates”

Clique em "Add New Template"

Escolha:

- Nome: ESP32-DHT11
- Hardware: ESP32
- Connection: WiFi

Clique em "Create"

Dentro do template, clique em "Datastreams"

Agora você pode adicionar os dois fluxos de dados:

V0 — Temperatura

- Name: Temperatura
- Virtual Pin: V0
- Data Type: Float
- Unit: °C
- Min: -10
- Max: 50

V1 — Umidade

- Name: Umidade
- Virtual Pin: V1
- Data Type: Float
- Unit: %
- Min: 0
- Max: 100

Vá para a aba "Web Dashboard"

- Arraste widgets como **Label**, **Chart**, etc.
- Clique duas vezes no widget e selecione o **Datastream (V0 ou V1)**

Clique em "Save" e depois "Apply Changes"

Crie um novo dispositivo baseado neste template:

- Vá para **"Devices"**
- Clique em **"New Device" > "From Template"**
- Escolha o Template que criou (ESP32 - DHT11)
- Nomeie e **crie**

O Blynk vai gerar um **AUTH TOKEN novo**. Copie ele e use no seu código:

Agora você verá os dados **chegando no Dashboard Web** corretamente, com total controle de:

- Pinagem (V0, V1, etc)

- Unidades, escala, tipo de dado (float, int, etc)
- Layout do painel

Código Exemplo de conexão com o Blynk.io com ESP32

```
#define BLYNK_TEMPLATE_ID "TMPL2GDECKUcG"
#define BLYNK_TEMPLATE_NAME "IoT Fatec"
#define BLYNK_AUTH_TOKEN "dgd3madcaMqVABaDSFRh xvugJUUX7-Bt"

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include "DHT.h"

char ssid[] = "Nome SSID";
char pass[] = "senha";

#define DHTPIN 5           // GPIO conectado ao pino de dados do DHT11
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

BlynkTimer timer;

void enviarSensor() {
  float temperatura = dht.readTemperature();
  float umidade = dht.readHumidity();

  if (isnan(temperatura) || isnan(umidade)) {
    Serial.println("Falha na leitura do DHT11");
    return;
  }

  Serial.print("Temperatura: ");
  Serial.print(temperatura);
  Serial.print(" °C | Umidade: ");
  Serial.println(umidade);

  // Envia para Blynk
  Blynk.virtualWrite(V0, temperatura);
  Blynk.virtualWrite(V1, umidade);
}

void setup() {
  Serial.begin(115200);
```

```
dht.begin();

Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

timer.setInterval(5000L, enviarSensor); // envia a cada 5 segundos
}

void loop() {
  Blynk.run();
  timer.run();
}
```

Para ESP8266 alterar as bibliotecas

```
#include <ESP8266WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp8266.h>
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT11
```